

Material instrucional

Testes E2E com IA e Playwright

Geração, execução e análise de testes ponta a ponta usando Python, pytest e Playwright

Objetivo do material

Este roteiro orienta a criação de testes E2E com apoio de IA. O foco é transformar um fluxo de usuário em um prompt contextualizado, gerar o teste com Playwright, executar com pytest e interpretar os resultados de forma crítica.

1. Visão geral da prática

Nesta prática, será criado um teste E2E para validar o fluxo de login em uma aplicação com páginas HTML locais. O teste deve simular ações reais do usuário: abrir a tela, preencher credenciais, clicar no botão de login, verificar o redirecionamento e confirmar que o botão de logout está visível no dashboard.

Etapa	Descrição
1. Analisar o fluxo	Entender a jornada que será validada do ponto de vista do usuário.
2. Mapear seletores	Identificar os elementos da interface que serão usados pelo teste.
3. Criar prompt contextualizado	Informar à IA ferramenta, fluxo, dados, seletores e validações esperadas.
4. Gerar o teste	Criar o arquivo de teste com Playwright, Python e pytest.
5. Executar e analisar	Rodar o teste, verificar falhas e corrigir problemas de implementação ou de automação.

2. O que é um teste E2E

Teste E2E, ou End-to-End, valida um fluxo completo da aplicação a partir da perspectiva do usuário. Em vez de testar apenas uma função isolada, o teste abre uma tela, interage com elementos visuais, realiza ações e valida o resultado final esperado.

- Testes unitários validam funções e regras isoladas.
- Testes de API validam contratos de endpoints, status code, headers e respostas.
- Testes E2E validam jornadas completas, como login, busca, carrinho, pagamento ou finalização de compra.

Ponto de atenção

Testes E2E são mais próximos do uso real do sistema, mas também são mais lentos e mais sensíveis a mudanças de interface. Por isso, devem ser usados em fluxos críticos e bem definidos.

3. Ferramentas utilizadas

A prática utiliza Playwright com Python e pytest. O Playwright automatiza o navegador e o pytest executa o teste, apresenta resultados e indica falhas.

Ferramenta	Papel na prática
Python	Linguagem usada para escrever o teste automatizado.
pytest	Framework usado para executar os testes e apresentar os resultados.
Playwright	Framework usado para controlar o navegador e interagir com a interface.
pathlib.Path	Recurso usado para localizar os arquivos HTML locais de forma segura.
IA	Apoio para gerar o código do teste a partir de um prompt contextualizado.

4. Estrutura esperada do projeto

O projeto deve conter as telas HTML e o arquivo de teste. Como os arquivos são locais, as páginas serão abertas usando o protocolo file://, por meio de Path(__file__).parent e as_uri().

```
test_playwright/  
|  
|-- login.html  
|-- dashboard.html  
|-- test_login.py  
|-- requirements.txt
```

Arquivo	Responsabilidade
login.html	Tela de autenticação com campos de e-mail, senha e botão de login.
dashboard.html	Tela carregada após autenticação bem-sucedida.
test_login.py	Arquivo que contém o teste E2E automatizado.
requirements.txt	Arquivo com as dependências do projeto.

5. Contexto do desafio: fluxo de login

O primeiro desafio consiste em criar um teste E2E para validar o fluxo de login. O teste deve abrir a tela login.html, preencher e-mail e senha válidos, clicar no botão de login e verificar se a aplicação redirecionou para dashboard.html.

Item	Informação esperada
Tela inicial	login.html
Tela final	dashboard.html
Campo de e-mail	#email
Campo de senha	#senha
Botão de login	#btn-login

Botão de logout	#btn-logout
E-mail válido	user@test.com
Senha válida	123456
Resultado esperado	Redirecionamento para dashboard.html
Validação final	Botão #btn-logout visível

6. Como pensar em um teste E2E

Antes de pedir o teste para a IA, organize o fluxo em etapas. Isso reduz respostas genéricas e aumenta a chance de a IA gerar um teste executável.

1. Abrir a página inicial do fluxo.
2. Preencher os campos obrigatórios.
3. Executar a ação principal do usuário.
4. Validar a navegação para a página esperada.
5. Validar um elemento final que confirme o sucesso do fluxo.

Objetivo do teste

Confirmar que o fluxo de autenticação funciona do início ao fim, não apenas que o botão de login existe. O teste deve comprovar que o usuário chega ao dashboard e visualiza um elemento esperado da tela autenticada.

7. Elementos essenciais de um prompt para E2E

Parte do prompt	Finalidade
Contexto do projeto	Explica que o projeto usa arquivos HTML locais e como eles devem ser abertos.
Ferramenta	Define Playwright com Python e pytest, evitando que a IA gere código em outra stack.
Fluxo do usuário	Descreve as ações que o teste deve simular.
Seletores da interface	Informa quais elementos devem ser usados em <code>page.fill()</code> , <code>page.click()</code> e <code>expect()</code> .
Dados de entrada	Define credenciais e demais valores usados na execução.
Validações finais	Explicita como confirmar que o fluxo funcionou.
Restrições técnicas	Evita soluções frágeis, como curingas na validação de URL.

8. Prompt recomendado para o desafio de login

Use este prompt para gerar o arquivo `test_login.py`. Ele é contextualizado, informa a estrutura do projeto, os seletores, os dados de entrada e as validações esperadas.

Gere um teste E2E usando Playwright com Python e pytest para validar o fluxo de login.

Contexto:

O projeto usa arquivos HTML locais, abertos via file:///:

- login.html
- dashboard.html
- test_login.py

A tela login.html possui:

- #email
- #senha
- #btn-login

Após login válido, o usuário deve ser redirecionado para dashboard.html.

A tela dashboard.html possui o botão #btn-logout.

Dados de entrada:

- Email: user@test.com
- Senha: 123456

Validações:

- Abrir login.html usando Path(__file__).parent e login_path.as_uri()
- Validar a URL final usando dashboard_path.as_uri()
- Não use curingas como "**/dashboard.html" na validação da URL
- Validar que #btn-logout está visível

Requisitos:

- Use pathlib.Path
- Use Page e expect de playwright.sync_api
- Gere apenas o código do teste, sem explicações

9. Exemplo de teste esperado para login

O código abaixo representa uma solução de referência para o fluxo de login. Ele usa Path para localizar os arquivos HTML, as_uri() para abrir a página local e expect() para validar URL e visibilidade do botão de logout.

```
from pathlib import Path
from playwright.sync_api import Page, expect

def test_login_valido_redireciona_para_dashboard(page: Page):
    base_path = Path(__file__).parent
    login_path = base_path / "login.html"
    dashboard_path = base_path / "dashboard.html"

    page.goto(login_path.as_uri())
    page.fill("#email", "user@test.com")
    page.fill("#senha", "123456")
    page.click("#btn-login")

    expect(page).to_have_url(dashboard_path.as_uri())
    expect(page.locator("#btn-logout")).to_be_visible()
```

Objetivo do teste

Validar que credenciais corretas permitem a navegação da tela de login para o dashboard e que a área autenticada foi realmente carregada.

Motivo comum de falha

O teste pode falhar se algum seletor estiver diferente no HTML, se o redirecionamento não estiver implementado, se o arquivo dashboard.html não estiver no mesmo diretório ou se a validação de URL usar um caminho diferente do gerado por as_uri().

10. Instalação e execução

Execute os comandos abaixo no terminal, dentro da pasta do projeto. A instalação do Playwright inclui o pacote Python e os navegadores necessários para a automação.

```
python -m venv .venv
source .venv/bin/activate
pip install pytest pytest-playwright playwright
playwright install
pytest
```

No Windows

A ativação do ambiente virtual normalmente é feita com `.venv\Scripts\activate`. O restante dos comandos pode permanecer igual, desde que Python e pip estejam configurados corretamente.

11. Como interpretar o resultado do pytest

Resultado	Interpretação
1 passed	O fluxo foi executado e todas as validações passaram.
AssertionError	Alguma expectativa não foi atendida, como URL incorreta ou elemento não visível.
TimeoutError	O Playwright esperou por um elemento ou condição que não aconteceu no tempo limite.
Error: strict mode violation	O seletor encontrou mais de um elemento. Use um seletor mais específico.
FileNotFoundError ou URL inválida	O caminho do arquivo HTML pode estar incorreto.

12. Diagnóstico de falhas com IA

Depois de executar pytest, copie a mensagem de erro e peça uma análise para a IA. A IA pode ajudar a identificar se a falha está no teste, no seletor, no caminho do arquivo ou no comportamento da aplicação.

```
Análise a falha abaixo em um teste E2E com Playwright e pytest.
Explique:
1. Qual validação falhou;
2. Qual pode ser a causa provável;
3. Se o problema parece estar no teste, no HTML ou no fluxo da aplicação;
4. Como corrigir.

Erro:
[cole aqui a saída do pytest]
```

13. Desafio avançado: fluxo de compra

Depois do fluxo de login, avance para um fluxo maior. Este desafio valida uma jornada com login, busca de produto, tela de detalhes, adição ao carrinho e conferência do item no carrinho.

Arquivo/Tela	Função no fluxo
login.html	Tela inicial para autenticação.
produtos.html	Tela de listagem e busca de produtos.
produto.html	Tela de detalhes do produto selecionado.
carrinho.html	Tela final com o produto adicionado.
test_fluxo_compra.py	Arquivo do teste E2E do fluxo completo.

14. Prompt recomendado para o fluxo de compra

Gere um teste E2E usando Playwright com Python e pytest.

Contexto:

O projeto usa arquivos HTML locais abertos via file://.

Estrutura do projeto:

- login.html
- produtos.html
- produto.html
- carrinho.html
- test_fluxo_compra.py

Fluxo do usuário:

1. Acessar login.html
2. Preencher email e senha válidos
3. Clicar no botão de login
4. Ser redirecionado para produtos.html
5. Pesquisar por "Notebook"
6. Visualizar o produto Notebook na lista
7. Abrir a tela de detalhes do produto
8. Validar que produto.html foi carregado
9. Adicionar o produto ao carrinho
10. Validar que carrinho.html foi carregado
11. Confirmar que o produto Notebook aparece no carrinho

Seletores:

- Campo de email: #email
- Campo de senha: #senha
- Botão de login: #btn-login
- Campo de busca: #input-busca
- Botão de busca: #btn-buscar
- Produto Notebook na listagem: #produto-notebook
- Nome do produto na tela de detalhes: #produto-nome
- Botão adicionar ao carrinho: #btn-add-cart
- Item no carrinho: #item-carrinho

Dados de entrada:

- Email: user@test.com
- Senha: 123456
- Produto pesquisado: Notebook

Validações:

- Use Path(__file__).parent para localizar os arquivos HTML
- Use as_uri() para abrir login.html
- Valide as URLs finais usando produtos_path.as_uri(), produto_path.as_uri() e carrinho_path.as_uri()
- Não use curingas como "**/produtos.html", "**/produto.html" ou "**/carrinho.html"
- Valide que o produto Notebook aparece na lista
- Valide que o nome do produto na tela de detalhes é "Notebook"
- Valide que o item no carrinho é "Notebook"

Requisitos:

- Use pytest com Playwright em Python
- Importe Path de pathlib
- Importe Page e expect de playwright.sync_api
- Crie um nome claro para o teste
- Gere apenas o código do arquivo test_fluxo_compra.py, sem explicações

15. Exemplo de teste esperado para o fluxo de compra

```
from pathlib import Path
from playwright.sync_api import Page, expect

def test_fluxo_compra_notebook(page: Page):
    base_path = Path(__file__).parent
    login_path = base_path / "login.html"
    produtos_path = base_path / "produtos.html"
    produto_path = base_path / "produto.html"
    carrinho_path = base_path / "carrinho.html"

    page.goto(login_path.as_uri())
    page.fill("#email", "user@test.com")
    page.fill("#senha", "123456")
    page.click("#btn-login")

    expect(page).to_have_url(produtos_path.as_uri())

    page.fill("#input-busca", "Notebook")
    page.click("#btn-buscar")
    expect(page.locator("#produto-notebook")).to_be_visible()

    page.click("#produto-notebook")
    expect(page).to_have_url(produto_path.as_uri())
    expect(page.locator("#produto-nome")).to_have_text("Notebook")

    page.click("#btn-add-cart")
    expect(page).to_have_url(carrinho_path.as_uri())
    expect(page.locator("#item-carrinho")).to_have_text("Notebook")
```

Objetivo do teste

Validar uma jornada mais completa, passando por login, busca, detalhe do produto e carrinho. Esse teste protege um fluxo crítico de compra, não apenas uma tela isolada.

Motivo comum de falha

O teste pode falhar se a navegação entre páginas não estiver implementada, se os IDs dos elementos não corresponderem aos seletores informados, se a busca não atualizar a lista ou se o carrinho não persistir o produto selecionado.

16. Checklist de entrega

- Prompt contextualizado usado para gerar o teste E2E.
- Arquivo test_login.py ou test_fluxo_compra.py gerado.
- Execução com pytest realizada.
- Registro do resultado da execução: sucesso ou falha.

- Análise breve do resultado obtido.
- Correção ou justificativa caso o teste tenha falhado.

17. Checklist de qualidade do teste E2E

- O teste simula um fluxo real do usuário.
- Os seletores usados existem no HTML.
- As URLs são validadas com `as_uri()`, sem curingas genéricos.
- O teste valida o estado final do fluxo, não apenas o clique.
- O nome do teste descreve claramente o comportamento validado.
- A falha do teste ajuda a identificar qual parte do fluxo quebrou.
- O código gerado pela IA foi revisado antes da entrega.

18. Erros comuns e como evitar

Erro comum	Como evitar
Usar seletor incorreto	Compare os IDs do teste com os IDs reais do HTML.
Validar apenas o clique	Valide também URL final e elemento visível na tela de destino.
Usar curingas na URL	Use o caminho exato com <code>Path</code> e <code>as_uri()</code> .
Não instalar navegadores do Playwright	Execute <code>playwright install</code> antes do <code>pytest</code> .
Copiar código da IA sem revisar	Verifique imports, caminhos, nomes de arquivos e seletores.
Criar fluxo longo demais sem validações intermediárias	Valide cada etapa crítica antes de avançar para a próxima.

19. Síntese final

Um teste E2E bem construído deve representar uma jornada relevante do usuário e validar o resultado final do fluxo. A IA pode acelerar a geração do teste, mas a qualidade depende diretamente da clareza do prompt e da revisão crítica do código gerado.

Sempre que um teste falhar, analise a falha antes de alterar o código. A falha pode indicar problema no teste, no seletor, no HTML, no fluxo da aplicação ou na regra de negócio esperada.